NATIONAL ECONOMICS UNIVERSITY, HANOI
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

DATABASE SYSTEM MANAGEMENT

MIDTERM REPORT – EXAM 02

Topic: Auto Repair Garage Management Database

Database `garage_mini`

Instructor: Tran Duc Minh
Group: Group 6
Students: Bui Thi Khanh Linh – 11230756
Ninh Ngoc Anh – 11230734
Nguyen Thi Thu Phuong – 11230770
Repository: https://github.com/fuongthu/DBSM_GROUP_6

Hanoi, 7 November 2026

Contents

1 Introduction and Objectives **4**

2 Requirements Analysis and Business Rules **5**

3 Database Design **6**

3.1 Relationships 7

3.2 Data dictionary 7

3.3 Design decisions 8

4 Schema Implementation (`schema.sql`) **10**

4.1 `repair_orders` 10

4.2 `repair_reports` 12

4.3 `payments` 13

5 Sample Data and Integrity Verification (`seed.sql`) **15**

5.1 Record counts 15

5.2 Scenario coverage 15

5.3 Integrity checks 16

5.4 Constraint violation tests 21

6 Queries (`queries.sql`) **23**

6.1 Q1. Completed orders in January 2025 23

6.2 Q2. Orders per team in Q1/2025, including teams with none 23

6.3 Q3. Customers with an order that has no repair report (excluding Cancelled) . . 24

6.4 Q4. Average repair cost per Completed order, by team (all time) 24

6.5 Q5. Top 3 technicians by Completed orders in 2025 25

6.6 Q6. Total paid by each customer in 2025 (by `payment_date`) 26

6.7 Q7. Customers with no payment record at all 28

6.8 Q8. All customers ranked by total confirmed repair cost 28

6.9 Q9. Cancellation rate per technician in 2025 29

6.10 Q10. Customers with more than 3 Completed orders in 2025 30

7 Views (`views.sql`) **32**

7.1 `v_detail` – one row per repair order 32

7.2 `v_group_summary` – one row per team 33
7.3 `v_debt` – one row per customer 34

8 Conclusion and Lessons Learned **37**

References **38**

A Repository Structure **39**

B Sample Data: the 20 Repair Orders **39**

C Full `schema.sql` **40**

List of Figures

1 ERD of the `garage_mini` database. . . . 6
2 The six tables created by `schema.sql`. . . . 10
3 Structure and constraints of `repair_orders`. . . . 11
4 Structure and constraints of `repair_reports`. . . . 12
5 Structure and constraints of `payments`. . . . 13
6 Row counts against the required minimums. . . . 15
7 Foreign-key integrity check (Check 1). . . . 17
8 Status and repair-report consistency check (Check 2). . . . 18
9 Payment checks: a confirmed cost exists, and payments do not exceed cost (Checks 3 and 4). . . . 19
10 Cost, collected amount and outstanding balance per order (Check 5). . . . 20
11 Rejected inserts T1 and T2. . . . 22
12 Rejected inserts T3 to T8. . . . 22
13 Q1 – Completed orders in January 2025. . . . 23
14 Q2 – Orders per team in Q1/2025. . . . 24
15 Q3 – Customers with an order lacking a repair report. . . . 24
16 Q4 – Average cost per Completed order by team. . . . 25
17 Q5 – Top 3 technicians by Completed orders in 2025. . . . 26
18 Q6 – Amount paid per customer in 2025. . . . 27
19 Q7 – Customers without any payment. . . . 28
20 Q8 – Customers ranked by confirmed repair cost. . . . 29
21 Q9 – Cancellation rate per technician in 2025. . . . 30
22 Q10 – Customers with more than 3 Completed orders in 2025. . . . 31
23 `v_detail` – 20 orders, NULL cost where there is no report. . . . 33
24 `v_group_summary` – statistics per team. . . . 34
25 `v_debt` – cost, payments and outstanding debt per customer. . . . 36

1 Introduction and Objectives

This project delivers a relational database for a small auto repair garage. It contains six tables, 20 sample repair orders, ten reporting queries and three reporting views, all written for MySQL 8.0.

A garage receives repair orders, assigns each one to a main technician and records the outcome of the repair together with its confirmed cost. Customers may pay for a repair in several instalments. The database therefore tracks service costs and real cash payments separately, and it must report outstanding debt without counting a cost twice.

The objectives of the exam, and where this report addresses them, are:

- **DDL** – create the database, six tables, primary keys, foreign keys and integrity constraints (Section 4).
- **DML** – load sample data that covers every scenario named in the brief and verify integrity with queries (Section 5).
- **JOIN, GROUP BY and HAVING** – answer ten business questions with independent queries (Section 6).
- **Views** – expose a detailed order list, a per-team summary and a per-customer debt statement (Section 7).

Tools used: MySQL 8.0 and MySQL Workbench. The four deliverable files run in the order `schema.sql` → `seed.sql` → `queries.sql` → `views.sql` on a freshly created database.

2 Requirements Analysis and Business Rules

The brief defines six entities and a set of business rules. Each rule is enforced either by a constraint in `schema.sql` or by a verification query in `seed.sql`, because MySQL does not allow `CHECK` to read other tables and the brief does not ask for triggers.

#	Business rule (from the brief)	How it is enforced
1	One team has many technicians; each technician belongs to exactly one team	`technicians.team_id` NOT NULL + foreign key to `teams`
2	Each repair order belongs to exactly one customer and one main technician	`customer_id` and `technician_id` NOT NULL + two foreign keys
3	Valid statuses: Pending, Completed, Cancelled	`ENUM('Pending','Completed','Cancelled')` NOT NULL
4	License plate must not be blank	NOT NULL + `CHECK` that the value contains a non-space character
5	An order has at most one repair report	`UNIQUE (order_id)` on `repair_reports` + foreign key
6	Cost is mandatory and not negative	`cost DECIMAL(12,2) NOT NULL + CHECK (cost >= 0)`
7	Only Completed orders have a report, and every Completed order has exactly one	Cross-table rule: verification query (Section 5.3, Check 2)
8	Each payment is a real cash-in with `amount > 0`, a date and a method (Cash, BankTransfer, Card)	`CHECK (amount > 0)`, NOT NULL columns, `ENUM` for method
9	Several payments per order are allowed	`payments.order_id` is a plain foreign key (not unique)
10	Money is collected only after a cost is confirmed	Cross-table rule: verification query (Check 3)
11	Total collected for an order must not exceed its confirmed cost	Aggregate rule: verification query (Check 4)
12	Debt = total confirmed cost − total money actually collected; a cost must not be counted once per payment	Cost and payments are aggregated separately in `v_debt`; Q8 does not join payments
13	Missing totals are shown as 0	`COALESCE(..., 0)` in queries and views
14	Annual reports use 2025; Q1 is 01/01/2025–31/03/2025	Half-open date ranges, e.g. `>= '2025-01-01' AND < '2025-04-01'`

Reporting conventions. Because `received_at` is a `DATETIME`, an order received at 23:30 on 31/03/2025 belongs to Q1. Every date filter therefore uses a half-open interval [start, next_start) instead of `BETWEEN ... AND '2025-03-31'`, which would silently drop that order.

3 Database Design

The design has six tables arranged as a chain `teams` → `technicians` → `repair_orders` ← `customers`, with `repair_reports` and `payments` hanging off each repair order.

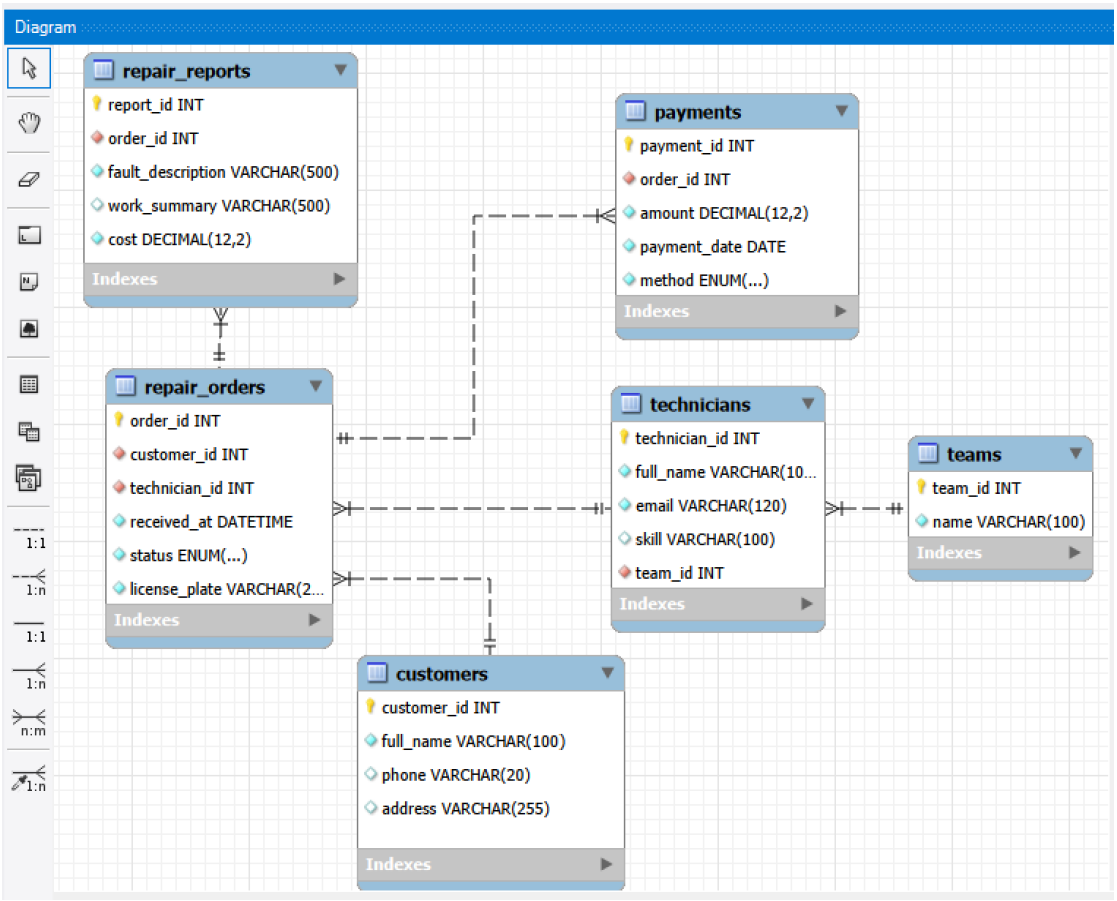


Figure 1: ERD of the `garage_mini` database.

3.1 Relationships

Parent	Child	Cardinality	Foreign key
teams	technicians	1:N – a technician belongs to exactly one team	team_id
customers	repair_orders	1:N – an order belongs to exactly one customer	customer_id
technicians	repair_orders	1:N – an order has exactly one main technician	technician_id
repair_orders	repair_reports	1:0..1 – at most one report per order	order_id (UNIQUE)
repair_orders	payments	1:N – several instalments per order	order_id

3.2 Data dictionary

teams – repair teams.

Column	Type	Constraints
team_id	INT, AUTO_INCREMENT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL, UNIQUE

technicians – staff who carry out repairs.

Column	Type	Constraints
technician_id	INT, AUTO_INCREMENT	PRIMARY KEY
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(120)	NOT NULL, UNIQUE
skill	VARCHAR(100)	NULL allowed
team_id	INT	NOT NULL, FOREIGN KEY → teams

customers – garage clients.

Column	Type	Constraints
customer_id	INT, AUTO_INCREMENT	PRIMARY KEY
full_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	NULL allowed
address	VARCHAR(255)	NULL allowed

repair_orders – one row per received vehicle (the “transaction” of the brief).

Column	Type	Constraints
order_id	INT, AUTO_INCREMENT	PRIMARY KEY
customer_id	INT	NOT NULL, FOREIGN KEY → customers
technician_id	INT	NOT NULL, FOREIGN KEY → technicians
received_at	DATETIME	NOT NULL (business date)
status	ENUM(Pending, Completed, Cancelled)	NOT NULL, default Pending
license_plate	VARCHAR(20)	NOT NULL, CHECK not blank

repair_reports – confirmed result and cost of a repair.

Column	Type	Constraints
report_id	INT, AUTO_INCREMENT	PRIMARY KEY
order_id	INT	NOT NULL, UNIQUE, FOREIGN KEY → repair_orders
fault_description	VARCHAR(500)	NOT NULL
work_summary	VARCHAR(500)	NULL allowed
cost	DECIMAL(12,2)	NOT NULL, CHECK (cost $\geq$ 0)

payments – real money received.

Column	Type	Constraints
payment_id	INT, AUTO_INCREMENT	PRIMARY KEY
order_id	INT	NOT NULL, FOREIGN KEY → repair_orders
amount	DECIMAL(12,2)	NOT NULL, CHECK (amount $>$ 0)
payment_date	DATE	NOT NULL
method	ENUM(Cash, BankTransfer, Card)	NOT NULL

3.3 Design decisions

- **Money as DECIMAL(12,2).** Floating-point types would introduce rounding errors in totals and debt; DECIMAL stores exact values [4].
- **DATETIME for received_at, DATE for payment_date.** An order has a time of arrival; a payment is a business day. The DATETIME column is the reason every range query is half-open.
- **ENUM for status and method.** The allowed values are fixed by the brief and MySQL rejects anything else [3].
- **Plate stored on the order.** The brief says plates are kept directly on the order, with no vehicle table and no ownership history.
- **No cascading deletes.** Foreign keys use the default RESTRICT, so a customer, technician or order with history cannot be deleted by accident.

- **No unique phone number.** The brief does not say that a phone number identifies a customer, and relatives or a company may share one, so no UNIQUE was added. Technician email is UNIQUE because it identifies a person.
- **Index choices.** InnoDB indexes every foreign key automatically; two extra indexes, on `received_at` and `payment_date`, support the date-range reports.
- **Cross-table rules are verified, not triggered.** The brief states that such rules may be verified by queries in `seed.sql`, so no triggers or stored procedures are used.

4 Schema Implementation (`schema.sql`)

`schema.sql` drops any previous `garage_mini`, recreates it with `utf8mb4` and creates the six tables in dependency order (parents first), so the file can be re-run on any machine. Because it *deletes* an existing `garage_mini`, it must only be run on a development or exam database, never on production data (the file carries the same warning as a comment).

```sql
-- WARNING: the next statement DELETES any existing database named garage_mini.
-- Run this file only on a development / exam database, never on production data.
DROP DATABASE IF EXISTS garage_mini;
CREATE DATABASE garage_mini
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE garage_mini;
```

The creation order is `teams`, `technicians`, `customers`, `repair_orders`, `repair_reports`, `payments`.

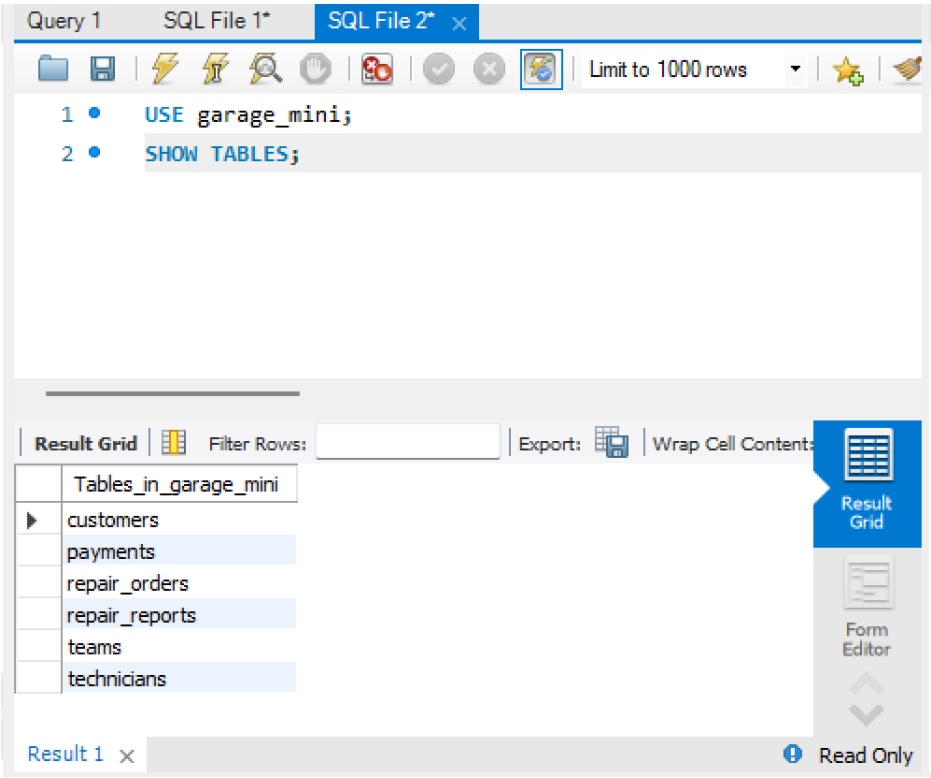


Figure 2: The six tables created by `schema.sql`.

4.1 `repair_orders`

This table holds the two foreign keys to the customer and the technician, the status `ENUM`, and the rule that a license plate cannot be blank.

```sql
CREATE TABLE repair_orders (
```

```
    order_id      INT          NOT NULL AUTO_INCREMENT,
    customer_id   INT          NOT NULL,
    technician_id INT          NOT NULL,
    received_at   DATETIME     NOT NULL,
    status        ENUM('Pending', 'Completed', 'Cancelled') NOT NULL DEFAULT 'Pending',
    license_plate VARCHAR(20) NOT NULL,
    CONSTRAINT pk_repair_orders PRIMARY KEY (order_id),
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id),
    CONSTRAINT fk_orders_technician FOREIGN KEY (technician_id)
        REFERENCES technicians (technician_id),
    CONSTRAINT chk_orders_plate_not_blank CHECK (license_plate REGEXP '[^[:space:]]')
) ENGINE = InnoDB;
```

The `CHECK` constraint [2] accepts a plate only if it contains at least one non-space character, so both an empty string and a string of spaces are rejected.

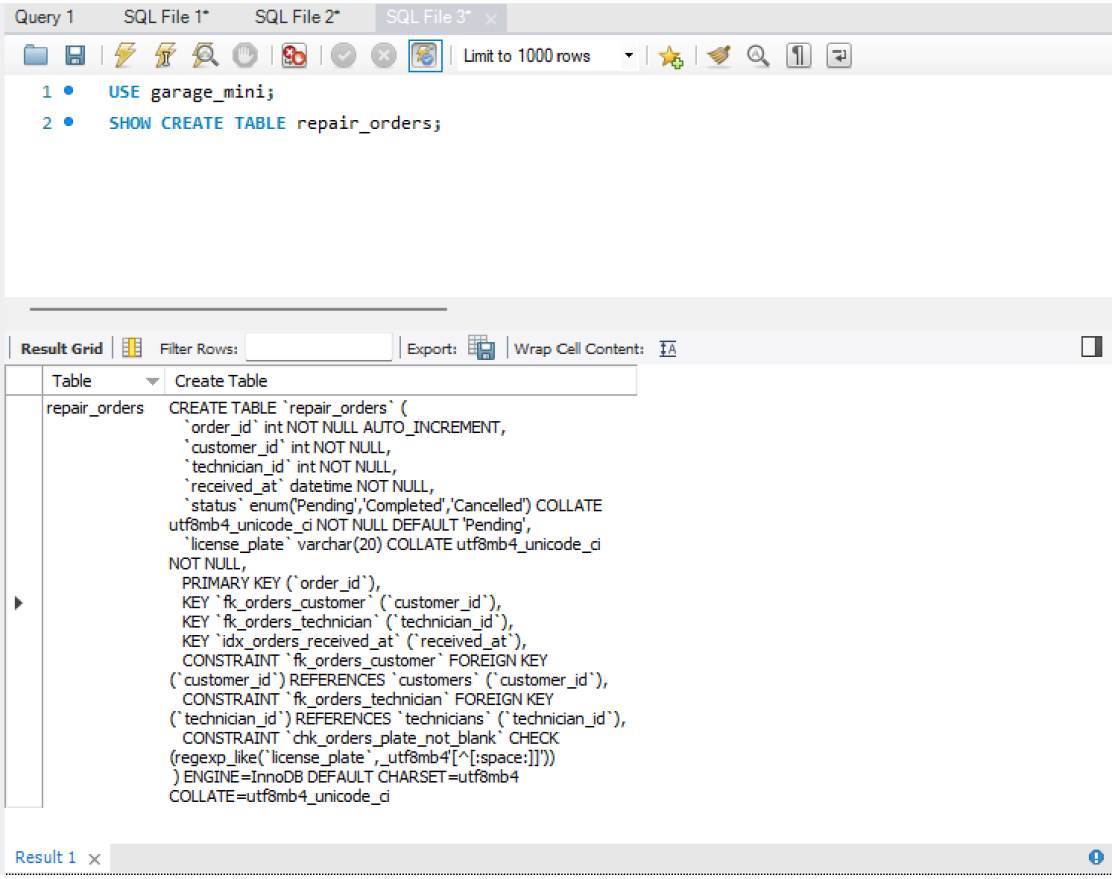


Figure 3: Structure and constraints of `repair_orders`.

4.2 repair_reports

The UNIQUE constraint on order_id is what limits an order to a single report; the CHECK keeps cost non-negative.

```
CREATE TABLE repair_reports (
    report_id         INT           NOT NULL AUTO_INCREMENT,
    order_id          INT           NOT NULL,
    fault_description VARCHAR(500)  NOT NULL,
    work_summary      VARCHAR(500)  NULL,
    cost              DECIMAL(12,2) NOT NULL,
    CONSTRAINT pk_repair_reports PRIMARY KEY (report_id),
    CONSTRAINT uq_reports_order UNIQUE (order_id),
    CONSTRAINT fk_reports_order FOREIGN KEY (order_id)
        REFERENCES repair_orders (order_id),
    CONSTRAINT chk_reports_cost CHECK (cost >= 0)
) ENGINE = InnoDB;
```

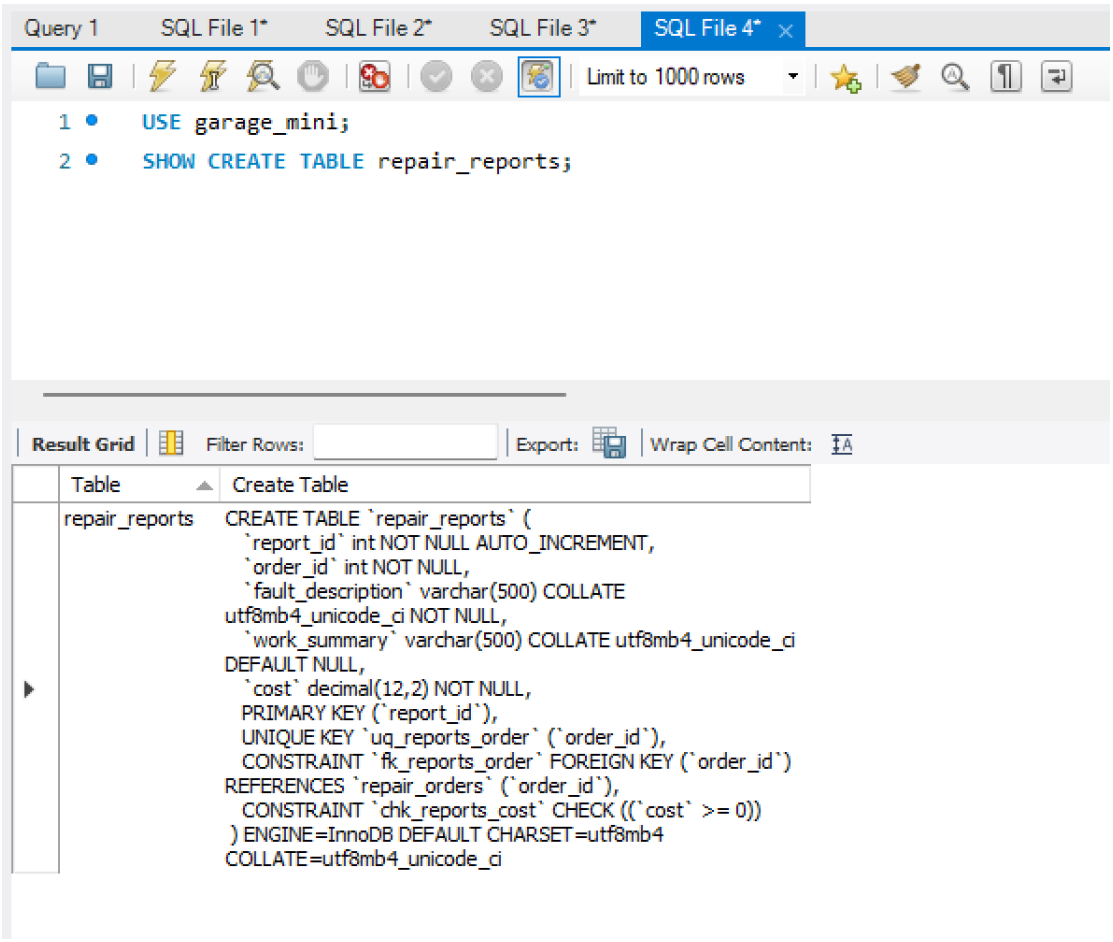


Figure 4: Structure and constraints of repair_reports.

4.3 payments

order_id is deliberately not unique here, so an order can be paid in several instalments; each row must be a positive amount.

```
CREATE TABLE payments (
    payment_id   INT           NOT NULL AUTO_INCREMENT,
    order_id     INT           NOT NULL,
    amount       DECIMAL(12,2) NOT NULL,
    payment_date DATE          NOT NULL,
    method       ENUM('Cash', 'BankTransfer', 'Card') NOT NULL,
    CONSTRAINT pk_payments PRIMARY KEY (payment_id),
    CONSTRAINT fk_payments_order FOREIGN KEY (order_id)
        REFERENCES repair_orders (order_id),
    CONSTRAINT chk_payments_amount CHECK (amount > 0)
) ENGINE = InnoDB;
```

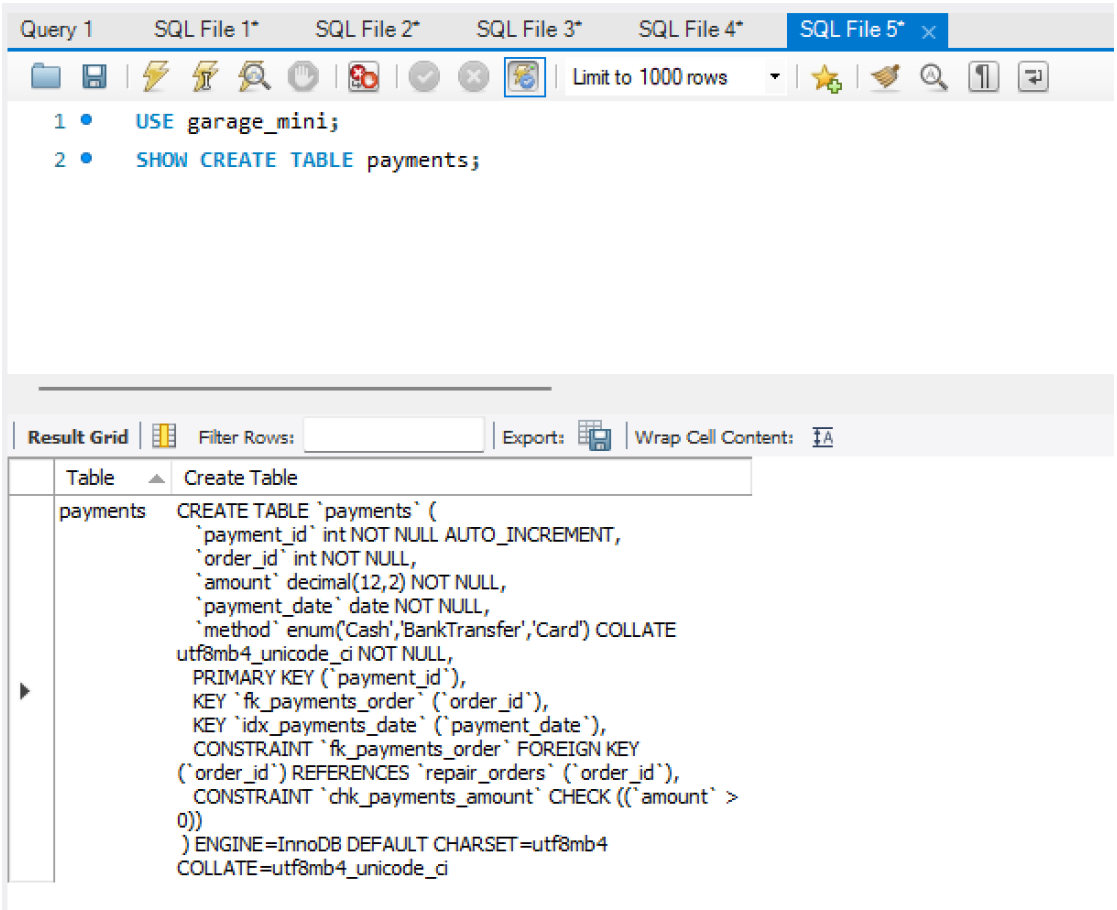


Figure 5: Structure and constraints of payments.

The remaining three tables (teams, technicians, customers) follow the data dictionary in Section 3.2; their full definitions are in schema.sql in the repository. Two extra indexes,

`idx_orders_received_at` and `idx_payments_date`, are created at the end of the file.

5 Sample Data and Integrity Verification (`seed.sql`)

`seed.sql` loads 20 repair orders and their related rows, then runs six verification queries. Every violation count returns 0, and the row counts exceed the minimums in the brief.

5.1 Record counts

Table	Required minimum	Loaded
`teams`	5	5
`technicians`	6	6
`customers`	10	10
`repair_orders`	12	20
`repair_reports`	10	15
`payments`	10	16

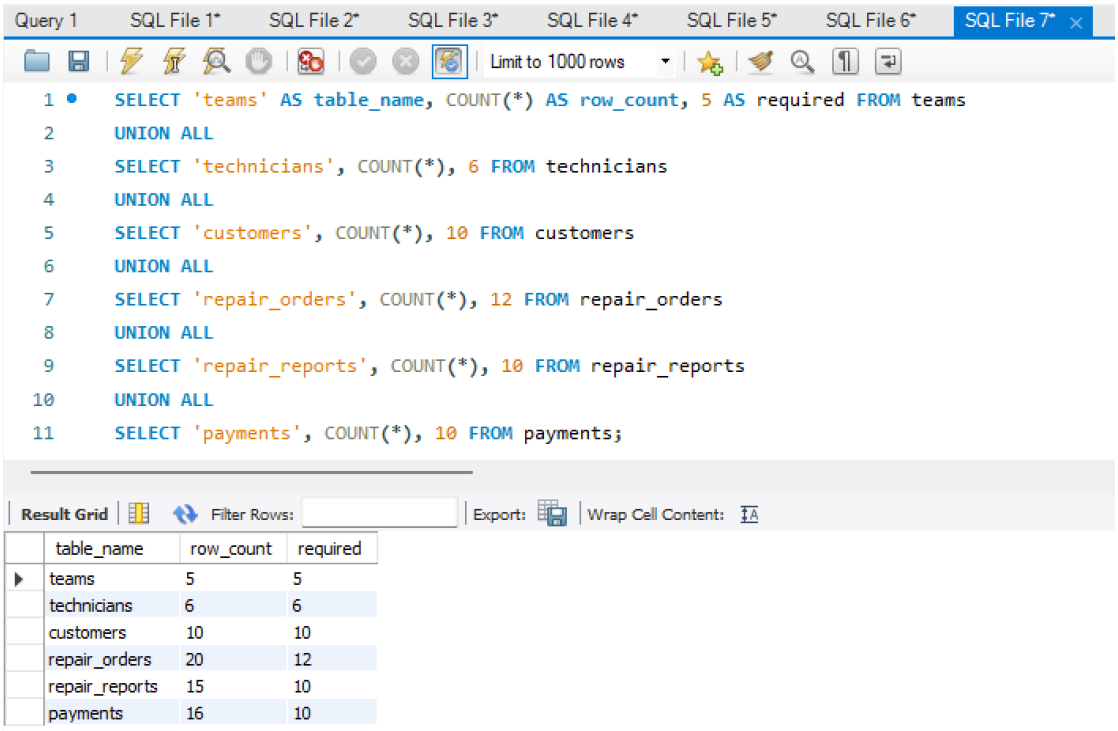


Figure 6: Row counts against the required minimums.

5.2 Scenario coverage

The data was designed so that each situation named in the brief appears at least once, and so that the usual query mistakes (lost zero rows, double counting, wrong date boundary) would

produce a visibly wrong result.

Scenario required by the brief	Where it appears in the data
Orders in January 2025	Orders 2, 3, 4, 5
Orders in the rest of Q1 2025	Orders 6, 7 (February); 8, 9, 10, 11 (March)
Orders outside Q1 but inside 2025	Orders 12–17, 19, 20
Orders outside 2025	Order 1 (November 2024); order 18 (January 2026)
Statuses Completed, Pending, Cancelled	Completed: e.g. 2, 3; Pending: 4, 15; Cancelled: 5, 11, 16
A customer with more than 3 Completed orders in 2025	Customer 1 (orders 2, 6, 9, 12 = 4)
A customer who has orders but never paid	Customer 4 (orders 5, 20) and customer 7 (orders 15, 16)
A customer with no orders at all	Customer 8
An order paid in several instalments	Order 2 (3 payments); order 9 (2 payments)
An order that is still owed	Orders 6, 7, 9 (partly paid); orders 12, 20 (unpaid)
A team with no order in Q1 2025	Team 5 (Air-Conditioning)
A technician with no order in 2025	Technician 6

Extra cases were added to stress the queries:

- **Boundary order.** Order 8 is received at 2025-03-31 23:30:00, the last evening of Q1, so a filter ending at `'2025-03-31'` would wrongly drop it.
- **Exactly three Completed orders.** Customers 2 and 3 each have 3 Completed orders in 2025, so "more than 3" must exclude them.
- **Payment date differs from order date.** Order 19 is received on 2025-12-28 but paid on 2026-01-05, so it counts in the 2025 order statistics but not in the 2025 payment total; order 8 is received in Q1 and paid on 2025-04-02.
- **Tie in the technician ranking.** Technicians 1, 2 and 4 each have 3 Completed orders in 2025, so the tie-break by `technician_id` decides places 2 and 3.
- **Team that never incurs cost.** Team 4 has only Pending and Cancelled orders, so it must be absent from Q4 but still present in `v_group_summary`.
- **Duplicate names.** Customers 2 and 10 are both called "Tran Van Hung", so results must show identifiers.

5.3 Integrity checks

The checks at the end of `seed.sql` implement the three verifications requested in the brief. Each returns a `violations` count that must be 0.

Check	What it verifies	Result
1	No orphan rows for any of the five foreign keys	0 for each key
2	No Completed order without a report; no non-Completed order with a report; no order with two reports	0 for each rule
3	No payment on an order that has no confirmed cost	0
4	No order whose total payments exceed its cost	0
5	Informational: cost, total paid, outstanding and payment count per order	see below

Check 1 uses `LEFT JOIN ... WHERE parent IS NULL`; Checks 2 and 3 use `LEFT JOIN` on `repair_reports`; Check 4 sums payments per order with `GROUP BY` and `HAVING SUM(p.amount) > r.cost`.

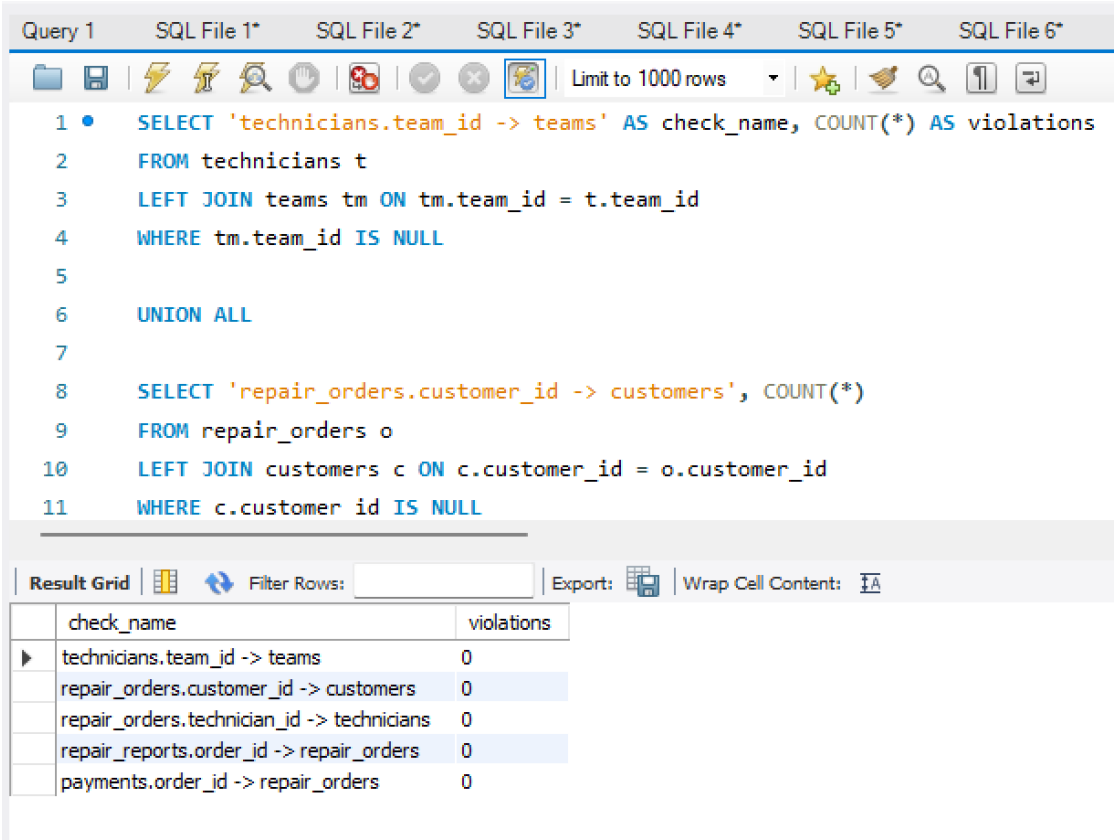


Figure 7: Foreign-key integrity check (Check 1).

SQL File 1* SQL File 2* SQL File 3* SQL File 4* SQL File 5* SQL File 6* SQL File 7*

Limit to 1000 rows

```
1  SELECT 'Completed order without report' AS check_name, COUNT(*) AS violations
2  FROM repair_orders o
3  LEFT JOIN repair_reports r ON r.order_id = o.order_id
4  WHERE o.status = 'Completed'
5    AND r.report_id IS NULL
6
7  UNION ALL
8
9  SELECT 'Non-Completed order with a report', COUNT(*)
10 FROM repair_orders o
11 JOIN repair reports r ON r.order id = o.order id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

check_name	violations
Completed order without report	0
Non-Completed order with a report	0
Order with more than one report	0

Figure 8: Status and repair-report consistency check (Check 2).

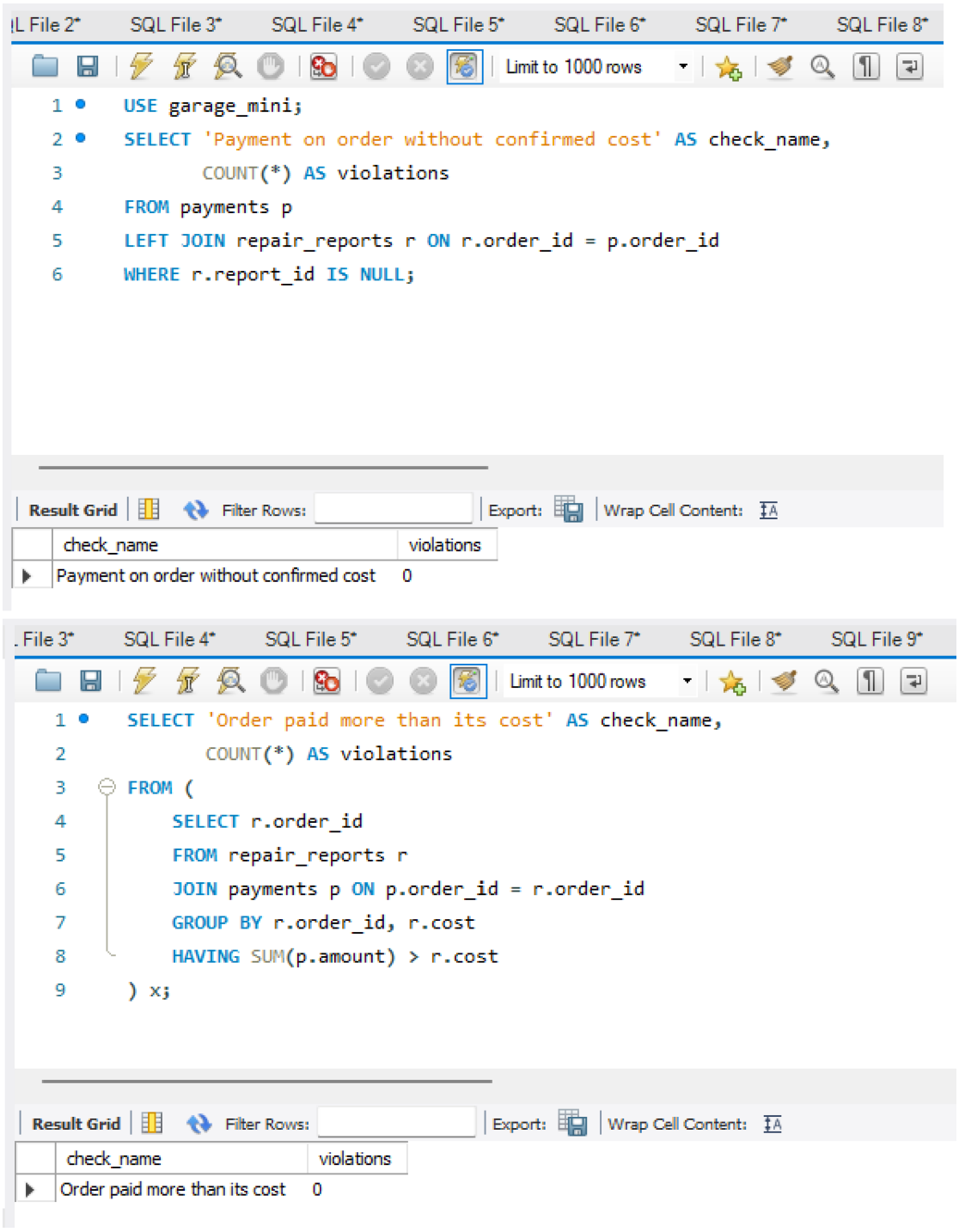


Figure 9: Payment checks: a confirmed cost exists, and payments do not exceed cost (Checks 3 and 4).

Check 5 lists each order with its cost, total paid and outstanding balance. The table below shows

the orders that have several payments or an outstanding balance.

order_id	cost	total_paid	outstanding	payment_count
2	4,500,000	4,500,000	0	3
6	6,000,000	3,000,000	3,000,000	1
7	1,800,000	1,000,000	800,000	1
9	12,500,000	9,000,000	3,500,000	2
12	3,300,000	0	3,300,000	0
20	2,400,000	0	2,400,000	0

The other nine orders that have a report (1, 3, 8, 10, 13, 14, 17, 18, 19) are fully paid with one payment each. Orders without a report (4, 5, 11, 15, 16) have no cost and cannot appear here.

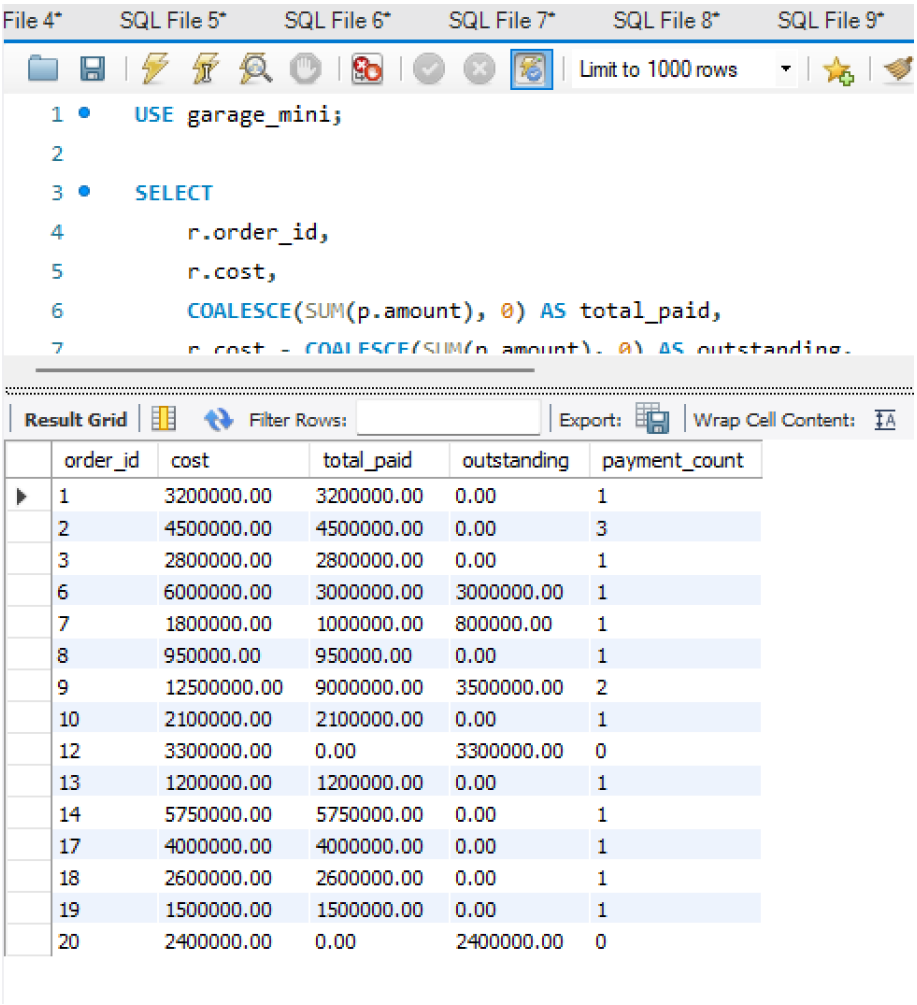


Figure 10: Cost, collected amount and outstanding balance per order (Check 5).

5.4 Constraint violation tests

The brief allows violating INSERT statements as long as they are kept out of the loaded files. They are kept in a separate evidence script (`evidence/extra_constraint_tests.sql`); each statement was run on its own, MySQL rejected all eight, and no row was inserted. All tests were run on MySQL 8.0 with the default `sql_mode` (strict mode enabled); in a non-strict mode the ENUM tests T2 and T7 would only raise a warning and store an empty value, so strict mode is required to reproduce them.

Test	Statement	Rule it breaks	MySQL error
T1	Order for customer 999	Foreign key `fk_orders_customer`	1452 – cannot add or update a child row
T2	Order with status `'Done'`	`status` ENUM	1265 – data truncated for column `status`
T3	Order with a blank license plate	`chk_orders_plate_not_blank`	3819 – check constraint violated
T4	Report with cost −100	`chk_reports_cost`	3819 – check constraint violated
T5	Second report for order 2	UNIQUE `uq_reports_order`	1062 – duplicate entry '2'
T6	Payment with amount 0	`chk_payments_amount`	3819 – check constraint violated
T7	Payment with method `'Crypto'`	`method` ENUM	1265 – data truncated for column `method`
T8	Technician with an existing email	UNIQUE `uq_technicians_email`	1062 – duplicate entry

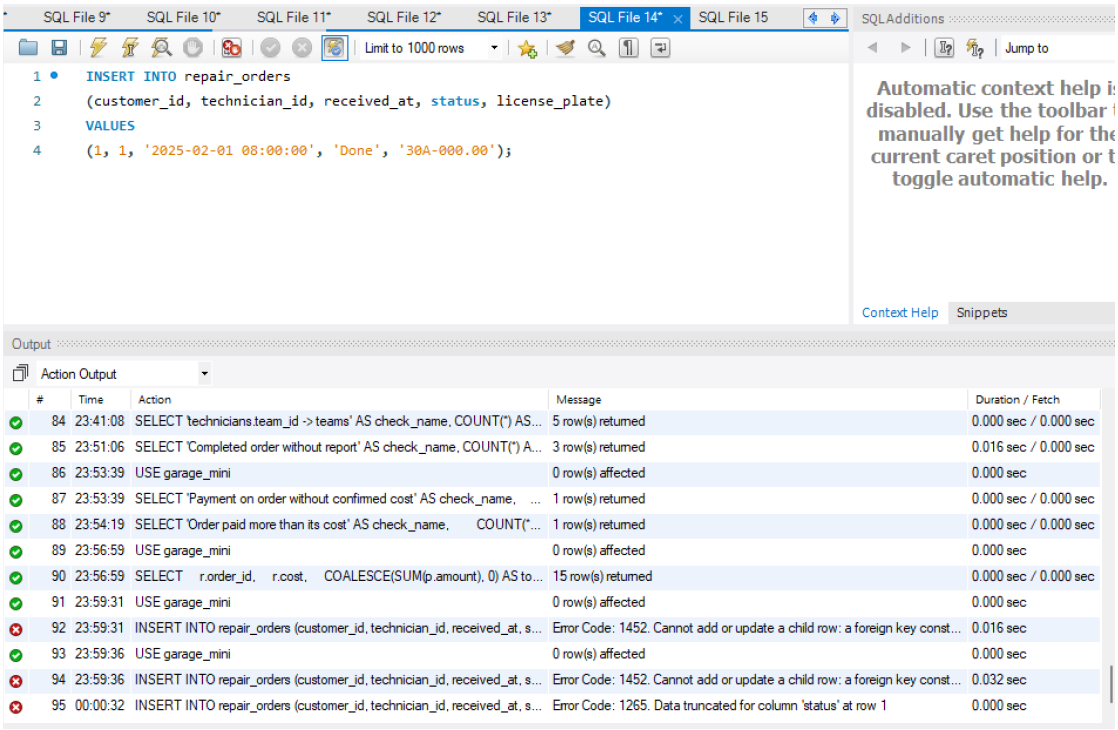


Figure 11: Rejected inserts T1 and T2.

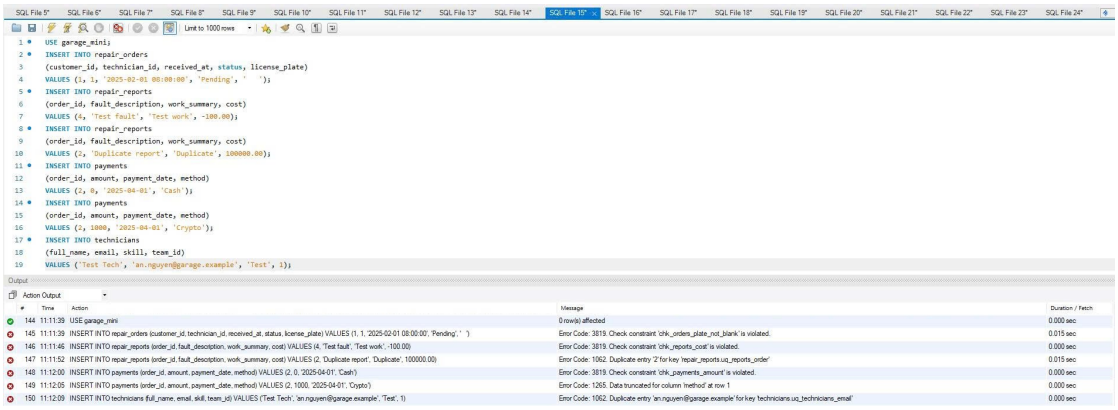


Figure 12: Rejected inserts T3 to T8.

6 Queries (queries.sql)

All ten queries run independently on the loaded data and return identifiers together with names, so that customers or technicians who share a name remain distinguishable. Three techniques recur:

- **Date filter inside ON.** When a query must keep rows that have no match (Q2, Q6, Q9), the date condition is placed in the `LEFT JOIN ... ON` clause. Placing it in `WHERE` would remove the unmatched rows and turn the outer join into an inner join [5].
- **Half-open date ranges.** `received_at >= '2025-01-01' AND received_at < '2025-04-01'` keeps the order received at 23:30 on 31/03/2025.
- **No join between cost and payments.** Q8 does not touch `payments`, so a cost cannot be multiplied by the number of instalments.

6.1 Q1. Completed orders in January 2025

```sql
SELECT o.order_id,
       o.received_at,
       c.customer_id,
       c.full_name      AS customer_name,
       t.technician_id,
       t.full_name      AS technician_name
FROM repair_orders o
JOIN customers   c ON c.customer_id   = o.customer_id
JOIN technicians t ON t.technician_id = o.technician_id
WHERE o.status = 'Completed'
  AND o.received_at >= '2025-01-01'
  AND o.received_at <  '2025-02-01'
ORDER BY o.received_at, o.order_id;
```

Two inner joins attach the customer and the technician to each Completed order of the month. **Result:** two rows – order 2 (customer 1, technician 1) and order 3 (customer 2, technician 3).

order_id	received_at	customer_id	customer_name	technician_id	technician_name
2	2025-01-08 08:30:00	1	Nguyen Thi Lan	1	Nguyen Van An
3	2025-01-15 10:00:00	2	Tran Van Hung	3	Le Thi Hoa

Figure 13: Q1 – Completed orders in January 2025.

6.2 Q2. Orders per team in Q1/2025, including teams with none

```sql
SELECT tm.team_id,
       tm.name AS team_name,
       COUNT(o.order_id) AS total_orders_q1_2025
FROM teams tm
LEFT JOIN technicians t ON t.team_id = tm.team_id
LEFT JOIN repair_orders o
       ON o.technician_id = t.technician_id
```

```
      AND o.received_at >= '2025-01-01'
      AND o.received_at <  '2025-04-01'
GROUP BY tm.team_id, tm.name
ORDER BY tm.team_id;
```

Both joins are `LEFT` and the date range sits in `ON`, so team 5 is listed with 0. `COUNT(o.order_id)` counts only real orders, never the NULL row of an unmatched team. All statuses are counted. **Result:** teams 1 to 5 have 4, 3, 2, 1 and 0 orders.

team_id	team_name	total_orders_q1_2025
1	Engine & Transmission Team	4
2	Electrical & Electronics Team	3
3	Body & Paint Team	2
4	Brakes & Suspension Team	1
5	Air-Conditioning Team	0

Figure 14: Q2 – Orders per team in Q1/2025.

6.3 Q3. Customers with an order that has no repair report (excluding Cancelled)

```
SELECT DISTINCT c.customer_id,
       c.full_name AS customer_name
FROM customers c
JOIN repair_orders o       ON o.customer_id = c.customer_id
LEFT JOIN repair_reports r ON r.order_id    = o.order_id
WHERE o.status <> 'Cancelled'
  AND r.report_id IS NULL
ORDER BY c.customer_id;
```

The anti-join (`LEFT JOIN ... WHERE r.report_id IS NULL`) finds orders without a report, and `DISTINCT` makes each customer appear once even with several such orders. Customer 4 is absent because order 5 is Cancelled and order 20 has a report. **Result:** customers 3 and 7.

customer_id	customer_name
3	Le Quang Vinh
7	Bui Anh Duc

Figure 15: Q3 – Customers with an order lacking a repair report.

6.4 Q4. Average repair cost per Completed order, by team (all time)

```
SELECT tm.team_id,
       tm.name AS team_name,
```

```
       COUNT(o.order_id)     AS completed_orders,
       ROUND(AVG(r.cost), 2) AS avg_cost_per_completed_order
FROM teams tm
JOIN technicians t    ON t.team_id       = tm.team_id
JOIN repair_orders o  ON o.technician_id = t.technician_id
                     AND o.status = 'Completed'
JOIN repair_reports r ON r.order_id      = o.order_id
GROUP BY tm.team_id, tm.name
ORDER BY tm.team_id;
```

Inner joins are used on purpose: a team without a Completed order that has a cost drops out, as the brief requires. Team 4 (Brakes & Suspension) therefore does not appear. **Result:** team 1: 3,816,666.67 (6 orders); team 2: 1,825,000.00 (4); team 3: 5,300,000.00 (4); team 5: 3,200,000.00 (1).

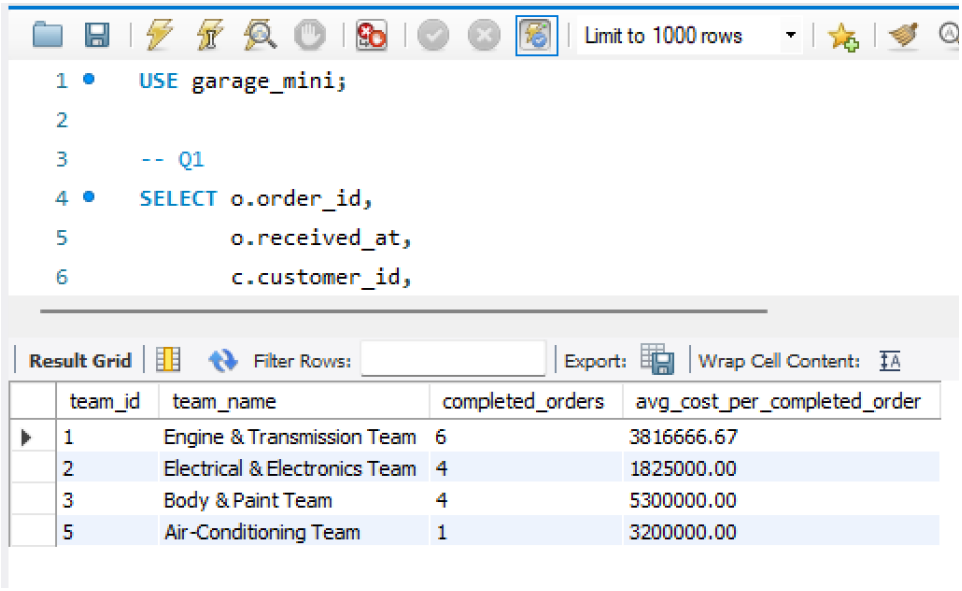


Figure 16: Q4 – Average cost per Completed order by team.

6.5 Q5. Top 3 technicians by Completed orders in 2025

```
SELECT t.technician_id,
       t.full_name AS technician_name,
       COUNT(o.order_id) AS completed_orders_2025
FROM technicians t
JOIN repair_orders o ON o.technician_id = t.technician_id
WHERE o.status = 'Completed'
  AND o.received_at >= '2025-01-01'
  AND o.received_at <  '2026-01-01'
GROUP BY t.technician_id, t.full_name
ORDER BY completed_orders_2025 DESC, t.technician_id ASC
LIMIT 3;
```

Technician 3 leads with 4. Technicians 1, 2 and 4 are tied at 3, and the secondary sort on `technician_id` gives places 2 and 3 to technicians 1 and 2; technician 4 is cut by `LIMIT 3`.

Technicians without orders cannot appear because of the inner join.

Result Grid | Filter Rows: | Export:

technician_id	technician_name	completed_orders_2025
3	Le Thi Hoa	4
1	Nguyen Van An	3
2	Tran Minh Tuan	3

Figure 17: Q5 – Top 3 technicians by Completed orders in 2025.

6.6 Q6. Total paid by each customer in 2025 (by `payment_date`)

```
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(SUM(p.amount), 0) AS total_paid_2025
FROM customers c
LEFT JOIN repair_orders o ON o.customer_id = c.customer_id
LEFT JOIN payments p      ON p.order_id    = o.order_id
                         AND p.payment_date >= '2025-01-01'
                         AND p.payment_date <  '2026-01-01'
GROUP BY c.customer_id, c.full_name
ORDER BY c.customer_id;
```

The year is taken from `payment_date`, not from the order date, and the filter is in `ON` so that customers without a 2025 payment stay in the result with 0. Customer 10 therefore shows 0 although the order was received in 2025: it was paid on 2026-01-05. Customer 5 also shows 0 because the only payment was in 2024. **Result:** customer 1: 16,500,000; customer 2: 9,750,000; customer 3: 7,050,000; customer 6: 1,000,000; all others 0.

SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9* SQL F

Limit to 1000 rows

```sql
USE garage_mini;
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(SUM(p.amount), 0) AS total_paid_2025
FROM customers c
LEFT JOIN repair_orders o ON o.customer_id = c.customer_id
LEFT JOIN payments p
       ON p.order_id = o.order_id
      AND p.payment_date >= '2025-01-01'
      AND p.payment_date < '2026-01-01'
GROUP BY c.customer_id, c.full_name
ORDER BY c.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_paid_2025
1	Nguyen Thi Lan	16500000.00
2	Tran Van Hung	9750000.00
3	Le Quang Vinh	7050000.00
4	Pham Thi Mai	0.00
5	Hoang Van Nam	0.00
6	Do Thi Thu	1000000.00
7	Bui Anh Duc	0.00
8	Vu Thi Hanh	0.00
9	Dang Minh Khoa	0.00
10	Tran Van Hung	0.00

Figure 18: Q6 – Amount paid per customer in 2025.

6.7 Q7. Customers with no payment record at all

```sql
SELECT c.customer_id,
       c.full_name AS customer_name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM repair_orders o
    JOIN payments p ON p.order_id = o.order_id
    WHERE o.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

`NOT EXISTS` returns every customer for whom no order has any payment, which includes customer 8, who has no order at all. Customers 5 and 10 are not listed because they did pay, even though not in 2025. **Result:** customers 4, 7 and 8.

customer_id	customer_name
4	Pham Thi Mai
7	Bui Anh Duc
8	Vu Thi Hanh

Figure 19: Q7 – Customers without any payment.

6.8 Q8. All customers ranked by total confirmed repair cost

```sql
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(SUM(r.cost), 0) AS total_confirmed_cost
FROM customers c
LEFT JOIN repair_orders o  ON o.customer_id = c.customer_id
LEFT JOIN repair_reports r ON r.order_id    = o.order_id
GROUP BY c.customer_id, c.full_name
ORDER BY total_confirmed_cost DESC, c.customer_id ASC;
```

The query joins customers → orders → reports only. Because `payments` is not joined and each order has at most one report, a cost is summed exactly once even when an order has three instalments. Customers with no cost show 0 and are sorted by `customer_id`. **Result:** customer 1 is first with 26,300,000; customers 7 and 8 are last with 0.

SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13*

Limit to 1000 rows

```
1  USE garage_mini;
2
3  -- Q1
4  SELECT o.order_id,
5         o.received_at,
6         c.customer_id,
7         c.full_name      AS customer_name,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_confirmed_cost
1	Nguyen Thi Lan	26300000.00
2	Tran Van Hung	9750000.00
3	Le Quang Vinh	7050000.00
5	Hoang Van Nam	3200000.00
9	Dang Minh Khoa	2600000.00
4	Pham Thi Mai	2400000.00
6	Do Thi Thu	1800000.00
10	Tran Van Hung	1500000.00
7	Bui Anh Duc	0.00
8	Vu Thi Hanh	0.00

Result 1 Result 2 Result 3 Result 4 Result 5 Result 6 customers 7 R

Figure 20: Q8 – Customers ranked by confirmed repair cost.

6.9 Q9. Cancellation rate per technician in 2025

```
SELECT t.technician_id,
       t.full_name AS technician_name,
       COUNT(o.order_id) AS total_orders_2025,
       COALESCE(SUM(CASE WHEN o.status = 'Cancelled' THEN 1 ELSE 0 END), 0) AS
    cancelled_orders_2025,
       ROUND(SUM(CASE WHEN o.status = 'Cancelled' THEN 1 ELSE 0 END) * 100.0
             / NULLIF(COUNT(o.order_id), 0), 2) AS cancellation_rate_pct
FROM technicians t
LEFT JOIN repair_orders o
       ON o.technician_id = t.technician_id
      AND o.received_at >= '2025-01-01'
      AND o.received_at <  '2026-01-01'
GROUP BY t.technician_id, t.full_name
ORDER BY t.technician_id;
```

`NULLIF(COUNT(...), 0)` turns a zero denominator into NULL, so technician 6, who has no 2025 order, gets a rate of NULL instead of a division error. The rate is rounded to two decimals. **Result:** 25.00, 0.00, 20.00, 0.00, 33.33 for technicians 1 to 5 and NULL for technician 6.

SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL Fi

Limit to 1000 rows

```
1  USE garage_mini;
2
3  -- Q1
4  SELECT o.order_id,
5         o.received_at,
6         c.customer_id,
7         c.full_name      AS customer_name,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

technician_id	technician_name	total_orders_2025	cancelled_orders_2025	cancellation_rate_pct
1	Nguyen Van An	4	1	25.00
2	Tran Minh Tuan	3	0	0.00
3	Le Thi Hoa	5	1	20.00
4	Pham Quoc Bao	3	0	0.00
5	Vo Thanh Long	3	1	33.33
6	Dang Huu Phuc	0	0	NULL

Figure 21: Q9 – Cancellation rate per technician in 2025.

6.10 Q10. Customers with more than 3 Completed orders in 2025

```sql
SELECT c.customer_id,
       c.full_name AS customer_name,
       COUNT(o.order_id) AS completed_orders_2025
FROM customers c
JOIN repair_orders o ON o.customer_id = c.customer_id
WHERE o.status = 'Completed'
  AND o.received_at >= '2025-01-01'
  AND o.received_at <  '2026-01-01'
GROUP BY c.customer_id, c.full_name
HAVING COUNT(o.order_id) > 3
ORDER BY completed_orders_2025 DESC, c.customer_id;
```

`WHERE` filters rows (Completed, in 2025) before grouping, and `HAVING` filters the groups afterwards. Customers 2 and 3 have exactly 3 Completed orders and are correctly excluded. **Result:** customer 1 with 4 orders.

Result Grid | Filter Rows: | Export:

customer_id	customer_name	completed_orders_2025
1	Nguyen Thi Lan	4

Figure 22: Q10 – Customers with more than 3 Completed orders in 2025.

7 Views (`views.sql`)

Three views cover the whole data set; no year is hard-coded. Each is created with `CREATE OR REPLACE VIEW` [6], so `views.sql` can be re-run, and each is followed by a `SELECT` that displays it.

7.1 `v_detail` – one row per repair order

```
CREATE OR REPLACE VIEW v_detail AS
SELECT o.order_id,
       o.received_at,
       o.status,
       c.customer_id,
       c.full_name     AS customer_name,
       t.technician_id,
       t.full_name     AS technician_name,
       tm.team_id,
       tm.name         AS team_name,
       r.cost          AS repair_cost
FROM repair_orders o
JOIN customers   c  ON c.customer_id   = o.customer_id
JOIN technicians t  ON t.technician_id = o.technician_id
JOIN teams       tm ON tm.team_id      = t.team_id
LEFT JOIN repair_reports r ON r.order_id = o.order_id;
```

Customer, technician and team always exist for an order, so they use inner joins. The report is joined with `LEFT JOIN`, so the view returns all 20 orders; the five orders without a report (4, 5, 11, 15, 16 – Pending or Cancelled) show `repair_cost = NULL`. Because a report is unique per order, no row is duplicated.

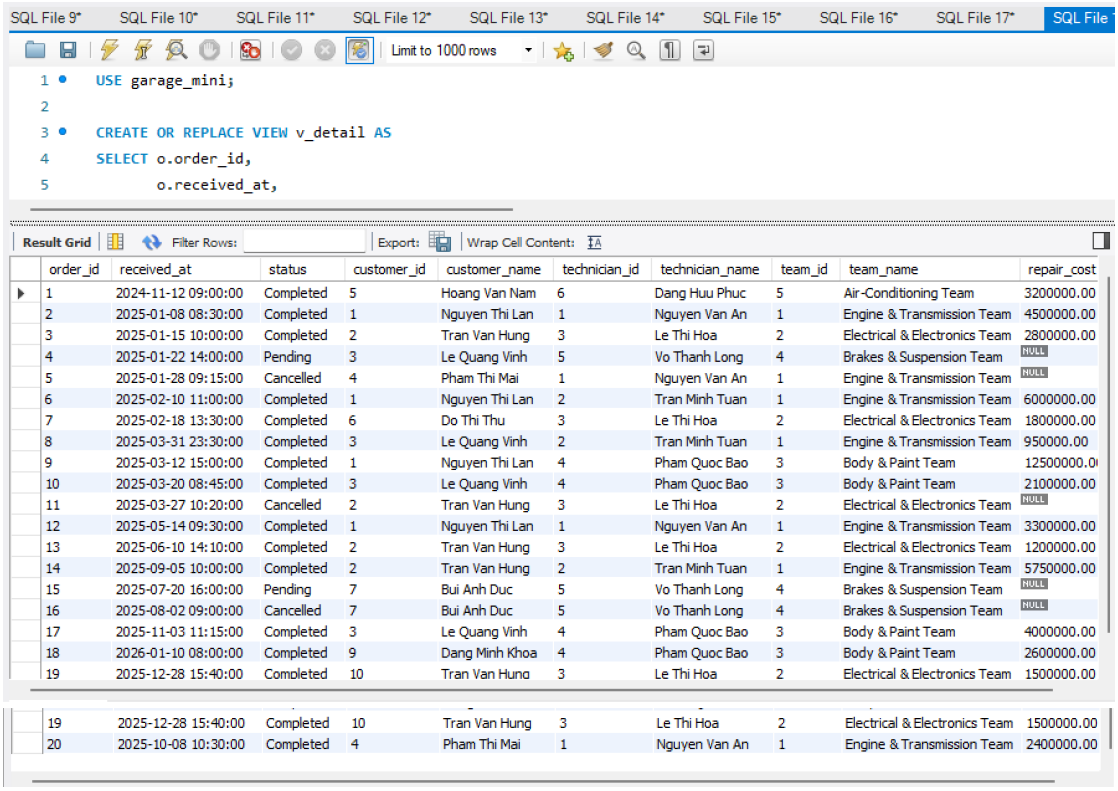


Figure 23: `v_detail` – 20 orders, NULL cost where there is no report.

7.2 `v_group_summary` – one row per team

```sql
CREATE OR REPLACE VIEW v_group_summary AS
SELECT tm.team_id,
       tm.name AS team_name,
       COUNT(o.order_id) AS total_orders,
       COALESCE(SUM(CASE WHEN o.status = 'Completed' THEN 1 ELSE 0 END), 0) AS completed_orders,
       COALESCE(SUM(CASE WHEN o.status = 'Cancelled' THEN 1 ELSE 0 END), 0) AS cancelled_orders,
       COALESCE(SUM(r.cost), 0) AS total_repair_cost
FROM teams tm
LEFT JOIN technicians t    ON t.team_id       = tm.team_id
LEFT JOIN repair_orders o  ON o.technician_id = t.technician_id
LEFT JOIN repair_reports r ON r.order_id      = o.order_id
GROUP BY tm.team_id, tm.name;
```

All joins start from `teams` with `LEFT JOIN`, so a team with no order would still appear, and `COALESCE` shows its totals as 0. Payments are not joined, so no cost is repeated.

team_id	team_name	total	completed	cancelled	total cost
1	Engine & Transmission Team	7	6	1	22,900,000
2	Electrical & Electronics Team	5	4	1	7,300,000
3	Body & Paint Team	4	4	0	21,200,000
4	Brakes & Suspension Team	3	0	1	0
5	Air-Conditioning Team	1	1	0	3,200,000

Team 4 counts its 3 orders (2 Pending, 1 Cancelled) in `total_orders`, but it has no Completed order, so its cost is 0.

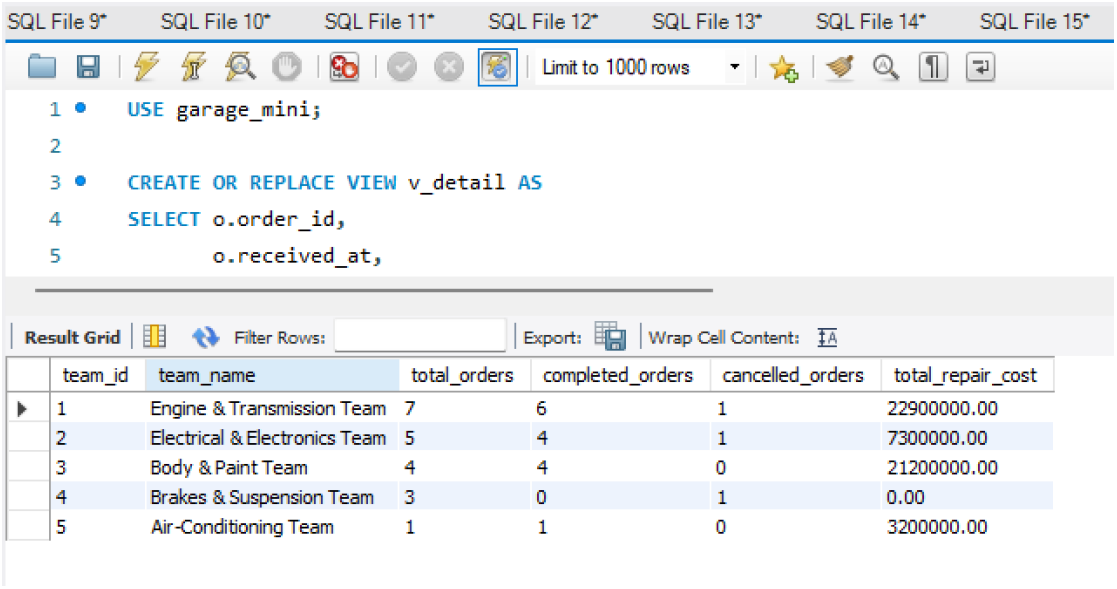


Figure 24: `v_group_summary` – statistics per team.

7.3 `v_debt` – one row per customer

```
CREATE OR REPLACE VIEW v_debt AS
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(cost.total_cost, 0) AS total_confirmed_cost,
       COALESCE(paid.total_paid, 0) AS total_paid,
       COALESCE(cost.total_cost, 0) - COALESCE(paid.total_paid, 0) AS outstanding_debt
FROM customers c
LEFT JOIN (
    SELECT o.customer_id, SUM(r.cost) AS total_cost
    FROM repair_orders o
    JOIN repair_reports r ON r.order_id = o.order_id
    GROUP BY o.customer_id
) cost ON cost.customer_id = c.customer_id
LEFT JOIN (
    SELECT o.customer_id, SUM(p.amount) AS total_paid
    FROM repair_orders o
```

```
    JOIN payments p ON p.order_id = o.order_id
    GROUP BY o.customer_id
) paid ON paid.customer_id = c.customer_id;
```

Why two subqueries. Joining `repair_reports` and `payments` in one query would repeat an order's cost once per payment. Order 2 (cost 4,500,000, three payments) would then count as 13,500,000 and the debt would be wrong. Here cost and payments are each summed per customer on their own and only then joined on `customer_id`, so each cost is counted once. Customers with no cost or no payment keep their row and show 0.

id	customer	confirmed cost	paid	debt
1	Nguyen Thi Lan	26,300,000	16,500,000	9,800,000
2	Tran Van Hung	9,750,000	9,750,000	0
3	Le Quang Vinh	7,050,000	7,050,000	0
4	Pham Thi Mai	2,400,000	0	2,400,000
5	Hoang Van Nam	3,200,000	3,200,000	0
6	Do Thi Thu	1,800,000	1,000,000	800,000
7	Bui Anh Duc	0	0	0
8	Vu Thi Hanh	0	0	0
9	Dang Minh Khoa	2,600,000	2,600,000	0
10	Tran Van Hung	1,500,000	1,500,000	0

The debt of customer 1 equals the sum of the three open orders: 3,000,000 (order 6) + 3,500,000 (order 9) + 3,300,000 (order 12) = 9,800,000. Customer 4 owes order 20 (2,400,000) and customer 6 owes 800,000 on order 7. Across all customers the confirmed cost is 54,600,000 and the money collected is 41,600,000, which leaves 13,000,000 outstanding (9,800,000 + 2,400,000 + 800,000) across the three debtors.

SQL File 9* SQL File 10* SQL File 11* SQL File 12* SQL File 13* SQL File 14* SQL File 15*

Limit to 1000 rows

```
1  USE garage_mini;
2
3  CREATE OR REPLACE VIEW v_detail AS
4  SELECT o.order_id,
5         o.received_at,
6         o.status,
7         c.customer_id,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_confirmed_cost	total_paid	outstanding_debt
1	Nguyen Thi Lan	26300000.00	16500000.00	9800000.00
2	Tran Van Hung	9750000.00	9750000.00	0.00
3	Le Quang Vinh	7050000.00	7050000.00	0.00
4	Pham Thi Mai	2400000.00	0.00	2400000.00
5	Hoang Van Nam	3200000.00	3200000.00	0.00
6	Do Thi Thu	1800000.00	1000000.00	800000.00
7	Bui Anh Duc	0.00	0.00	0.00
8	Vu Thi Hanh	0.00	0.00	0.00
9	Dang Minh Khoa	2600000.00	2600000.00	0.00
10	Tran Van Hung	1500000.00	1500000.00	0.00

v_detail 1 v_group_summary 2 v_debt 3

Figure 25: `v_debt` – cost, payments and outstanding debt per customer.

8 Conclusion and Lessons Learned

The `garage_mini` database meets every requirement of Exam 02: six normalized tables with primary and foreign keys, 20 orders that cover all required scenarios, ten queries, three views, and verification queries that return zero violations. The four files run in order on an empty database without errors, and each result was checked by hand against the sample data.

Lessons learned

- **Outer joins need care with filters.** A date condition in `WHERE` silently removes the rows that outer joins are meant to keep; moving it into `ON` fixed Q2, Q6 and Q9.
- **Joining two many-side tables multiplies rows.** Cost and payments both hang off an order, so joining them directly repeats the cost per payment. Aggregating each side separately (`v_debt`) or not joining payments at all (Q8, `v_group_summary`) avoids the error.
- **DATETIME needs half-open ranges.** The order at 23:30 on 31/03/2025 would be lost by `BETWEEN '2025-01-01' AND '2025-03-31'`.
- **Deliberate test data finds bugs.** Exact-3 customers, a three-way tie, a payment dated after its order's year and a team without cost each expose a specific mistake that a plain data set would hide.

Limitations and possible extensions

- Rules that span tables (a Completed order has exactly one report; money is collected only after a cost exists; total payments never exceed cost) are verified by queries, not enforced at insert time, as the brief allows. A production system could enforce them with triggers or stored procedures.
- Refunds and advance payments are out of scope by design, so the debt formula never goes negative.
- Vehicles are not modelled; the license plate is a plain attribute of the order.

References

[1] *Midterm Practical Examination, Exam 02: Auto Repair Garage Management (database `garage_mini`)*, Group 6. Examination brief supplied by the lecturer (`De_2_Gr6_Midterm.pdf`).

[2] Oracle Corporation. *MySQL 8.0 Reference Manual, 15.1.20.6 CHECK Constraints*. https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html (accessed 6 November 2026).

[3] Oracle Corporation. *MySQL 8.0 Reference Manual, 13.3.5 The ENUM Type*. https://dev.mysql.com/doc/refman/8.0/en/enum.html (accessed 6 November 2026).

[4] Oracle Corporation. *MySQL 8.0 Reference Manual, 13.1.3 Fixed-Point Types (Exact Value) – DECIMAL, NUMERIC*. https://dev.mysql.com/doc/refman/8.0/en/fixed-point-types.html (accessed 6 November 2026).

[5] Oracle Corporation. *MySQL 8.0 Reference Manual, 15.2.13.2 JOIN Clause* (inner and outer joins, `ON` conditions). https://dev.mysql.com/doc/refman/8.0/en/join.html (accessed 6 November 2026).

[6] Oracle Corporation. *MySQL 8.0 Reference Manual, 15.1.23 CREATE VIEW Statement*. https://dev.mysql.com/doc/refman/8.0/en/create-view.html (accessed 6 November 2026).

A Repository Structure

```
garage_mini/
|-- schema.sql      -- database, tables and constraints (run 1st)
|-- seed.sql        -- sample data and integrity checks (run 2nd)
|-- queries.sql     -- queries Q1 to Q10 (run 3rd)
|-- views.sql       -- v_detail, v_group_summary, v_debt (run 4th)
|-- report.pdf      -- this report
`-- evidence/
    `-- extra_constraint_tests.sql  -- statements expected to fail
```

The four root files run in the order shown on a fresh MySQL 8.0 server. The script in `evidence/` is deliberately outside that sequence because every statement in it is expected to fail.

B Sample Data: the 20 Repair Orders

The table lists every repair order in `seed.sql` together with the confirmed cost (from `repair_reports`) and the total collected (from `payments`), so that a reader can check the query results by hand. "–" means that the order has no report, so it has no confirmed cost. The data confirm, for example, the boundary order 8 received at 23:30 on 31/03/2025, which belongs to Q1.

Order	Cust.	Tech.	Received at	Status	Cost	Paid	Note
1	5	6	2024-11-12 09:00	Completed	3,200,000	3,200,000	Outside 2025 (Nov 2024)
2	1	1	2025-01-08 08:30	Completed	4,500,000	4,500,000	Q1; 3 instalments; Q1 January
3	2	3	2025-01-15 10:00	Completed	2,800,000	2,800,000	Q1; January
4	3	5	2025-01-22 14:00	Pending	–	0	No report yet
5	4	1	2025-01-28 09:15	Cancelled	–	0	Customer 4 never pays
6	1	2	2025-02-10 11:00	Completed	6,000,000	3,000,000	Partly paid (debt 3,000,000)
7	6	3	2025-02-18 13:30	Completed	1,800,000	1,000,000	Partly paid (debt 800,000)
8	3	2	2025-03-31 23:30	Completed	950,000	950,000	Boundary: last evening of Q1
9	1	4	2025-03-12 15:00	Completed	12,500,000	9,000,000	2 instalments; debt 3,500,000
10	3	4	2025-03-20 08:45	Completed	2,100,000	2,100,000	Q1
11	2	3	2025-03-27 10:20	Cancelled	–	0	Q1; cancelled
12	1	1	2025-05-14 09:30	Completed	3,300,000	0	Unpaid (debt 3,300,000)
13	2	3	2025-06-10 14:10	Completed	1,200,000	1,200,000	–
14	2	2	2025-09-05 10:00	Completed	5,750,000	5,750,000	–
15	7	5	2025-07-20 16:00	Pending	–	0	Customer 7 never pays
16	7	5	2025-08-02 09:00	Cancelled	–	0	Customer 7 never pays
17	3	4	2025-11-03 11:15	Completed	4,000,000	4,000,000	–
18	9	4	2026-01-10 08:00	Completed	2,600,000	2,600,000	Outside 2025 (Jan 2026)
19	10	3	2025-12-28 15:40	Completed	1,500,000	1,500,000	Ordered 2025, paid 2026-01-05
20	4	1	2025-10-08 10:30	Completed	2,400,000	0	Unpaid (debt 2,400,000)

Totals: confirmed cost 54,600,000; collected 41,600,000; outstanding 13,000,000.

C Full `schema.sql`

The listing below is the complete `schema.sql` used for all results in this report (it is the same file as in the repository; the project folder keeps a copy in `sql/schema.sql`).

```sql
DROP DATABASE IF EXISTS garage_mini;
CREATE DATABASE garage_mini
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE garage_mini;

CREATE TABLE teams (
    team_id INT          NOT NULL AUTO_INCREMENT,
    name    VARCHAR(100) NOT NULL,
    CONSTRAINT pk_teams PRIMARY KEY (team_id),
    CONSTRAINT uq_teams_name UNIQUE (name)
) ENGINE = InnoDB;

CREATE TABLE technicians (
    technician_id INT          NOT NULL AUTO_INCREMENT,
    full_name     VARCHAR(100) NOT NULL,
    email         VARCHAR(120) NOT NULL,
    skill         VARCHAR(100) NULL,
    team_id       INT          NOT NULL,
    CONSTRAINT pk_technicians PRIMARY KEY (technician_id),
    CONSTRAINT uq_technicians_email UNIQUE (email),
    CONSTRAINT fk_technicians_team FOREIGN KEY (team_id)
        REFERENCES teams (team_id)
) ENGINE = InnoDB;

CREATE TABLE customers (
    customer_id INT          NOT NULL AUTO_INCREMENT,
    full_name   VARCHAR(100) NOT NULL,
    phone       VARCHAR(20)  NULL,
    address     VARCHAR(255) NULL,
    CONSTRAINT pk_customers PRIMARY KEY (customer_id)
) ENGINE = InnoDB;

CREATE TABLE repair_orders (
    order_id      INT         NOT NULL AUTO_INCREMENT,
    customer_id   INT         NOT NULL,
    technician_id INT         NOT NULL,
    received_at   DATETIME    NOT NULL,
    status        ENUM('Pending', 'Completed', 'Cancelled') NOT NULL DEFAULT 'Pending',
    license_plate VARCHAR(20) NOT NULL,
    CONSTRAINT pk_repair_orders PRIMARY KEY (order_id),
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id),
    CONSTRAINT fk_orders_technician FOREIGN KEY (technician_id)
        REFERENCES technicians (technician_id),
    CONSTRAINT chk_orders_plate_not_blank CHECK (license_plate REGEXP '[^[:space:]]')
) ENGINE = InnoDB;

CREATE TABLE repair_reports (
    report_id         INT           NOT NULL AUTO_INCREMENT,
    order_id          INT           NOT NULL,
    fault_description VARCHAR(500)  NOT NULL,
    work_summary      VARCHAR(500)  NULL,
    cost              DECIMAL(12,2) NOT NULL,
    CONSTRAINT pk_repair_reports PRIMARY KEY (report_id),
    CONSTRAINT uq_reports_order UNIQUE (order_id),
    CONSTRAINT fk_reports_order FOREIGN KEY (order_id)
        REFERENCES repair_orders (order_id),
    CONSTRAINT chk_reports_cost CHECK (cost >= 0)
) ENGINE = InnoDB;

CREATE TABLE payments (
    payment_id   INT           NOT NULL AUTO_INCREMENT,
    order_id     INT           NOT NULL,
    amount       DECIMAL(12,2) NOT NULL,
    payment_date DATE          NOT NULL,
```

```
67      method       ENUM('Cash', 'BankTransfer', 'Card') NOT NULL,
68      CONSTRAINT pk_payments PRIMARY KEY (payment_id),
69      CONSTRAINT fk_payments_order FOREIGN KEY (order_id)
70          REFERENCES repair_orders (order_id),
71      CONSTRAINT chk_payments_amount CHECK (amount > 0)
72  ) ENGINE = InnoDB;
73
74  CREATE INDEX idx_orders_received_at ON repair_orders (received_at);
75  CREATE INDEX idx_payments_date      ON payments (payment_date);
```